DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: SOFTWARE ENGINEERING **Subject code: CS T55**

Unit II

Software Project Management and Requirements Analysis: Responsibilities of a Software Project Manager – Project Planning – Metrics for Project Size Estimation – Empirical Estimation Techniques – COCOMO – Halstead's Software Science – Staffing Level Estimation – Scheduling – Organization and Team structures – Staffing – Risk Management – Software Configuration Management – Requirements Gathering and Analysis – Software Requirements specification – Formal System Specification – Axiomatic Specification - Algebraic Specification – 4GL

2 Marks:

1. What are the responsibilities of a software project manager?

- Project proposal writing,
- Project cost estimation,
- Scheduling,
- Project staffing,
- Project monitoring and control,
- Software configuration management,
- Risk management,
- Managerial report writing and presentations, etc.

2. List activities involved in project planning? (NOV 2018)

- Estimation:
  - Effort, cost, resource, and project duration
- Project scheduling:
- Staff organization:
  - staffing plans
- Risk handling:
  - identification, analysis, and abatement procedures

3. List metrics for project size estimation?

- Lines of code (LOC)
- Function point (FP)

4. List project estimation techniques?

- Three main approaches to estimation:
  - Empirical
  - Heuristic
  - Analytical

5. Define heuristic estimation technique or COCOMO?

Heuristic techniques assume that the characteristics to be estimated can be expressed in terms of some mathematical expression.

6. Define Hallstead's software science or analytical technique?

Analytical techniques derive the required results starting from certain simple assumptions.

7. List staffing level estimation works?

- Norden's Work
- Putnam's Work:
- Jensen Model

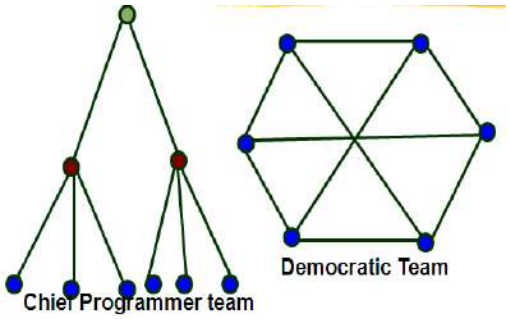
8. Write difference between Gantt chart and PERT chart?

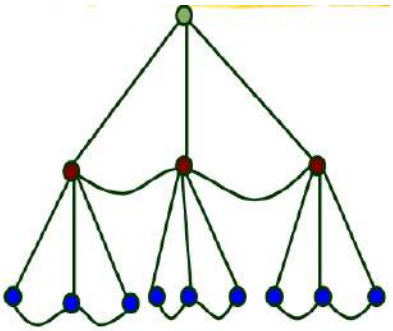
Gantt chart	PERT chart
A Gantt chart is a **special type of bar chart** where each bar represents an activity. The bars are drawn along a time line. The **length of each bar is proportional to the duration of time planned** for the corresponding activity.	The boxes of PERT charts are usually annotated with the **pessimistic, likely, and optimistic estimates** for every task.

9. Write differences between function format and project format in organization structure?

Functional Organization	Project Organization
Engineers are organized into functional groups, e.g. specification, design, coding, testing, maintenance, etc. **Engineers from functional groups get assigned to different projects**	**Engineers get assigned to a project for the entire duration of the project** Same set of engineers carry out all the phases

10. Give structures of chief programmer team, democratic team and mixed control team organization?





Mixed organization

11. Who is a good software engineer?

1. Exposure to systematic techniques, i.e. familiarity with software engineering principles
2. Good technical knowledge of the project areas (*domain knowledge*)
3. Good programming abilities
4. Good communication skills like oral, written and interpersonal skills
5. High motivation
6. Sound knowledge of fundamentals of computer science
7. Intelligence
8. Ability to work in a team
9. Discipline

12. What is Risk management and list activities in it? (NOV 2018)

- A risk is any anticipated unfavourable event or circumstance that can occur while a project is underway.
- Risk management aims at reducing the impact of all kinds of risks that might affect a project.
- It involves risk identification, risk assessment and risk mitigation.

13. How to compute risk leverage of the different risks?

$$\text{Risk leverage} = \frac{\text{Risk exposure before reduction} - \text{Risk exposure after reduction}}{\text{Cost of reduction}}$$

14. What is software configuration management (SCM)?

Software configuration management deals with effectively tracking and controlling the configuration of a software product during its life cycle.

15. List necessity of Software configuration management (SCM)?

- Inconsistency problem when the objects are replicated.
- Problems associated with concurrent access.
- Providing a stable development environment
- System accounting and maintaining status information. System accounting keeps
  - Track of who made a particular change and when the change was made.
- Handling variants**.** Existence of variants of a software product causes some peculiar problems.

16. What is the goal of the requirements analysis and specification phase?

The goal of the requirements analysis and specification phase is to clearly understand the customer requirements and to systematically organize the requirements into a specification document.

17. What are the ways the analyst gathers requirements?

Analyst gathers requirements through:

- observation of existing systems,
- studying existing procedures,
- discussion with the customer and end-users,
- Analysis of what needs to be done, etc.

18. What is the goal of requirements analysis?

The main purpose of the requirements analysis activity is to analyze the collected information to obtain a clear understanding of the product to be developed, with a view to removing all ambiguities, incompleteness, and inconsistencies from the initial customer perception of the problem.

19. Give difference between Model and Property-oriented methods?

In a model-oriented style, one defines **a system's behavior directly** by constructing a model of the system in terms of mathematical structures such as tuples, relations, functions, sets, sequences, etc.

In the property-oriented style, the **system's behavior is defined indirectly** by stating its properties, usually in the form of a set of axioms that the system must satisfy.

20. Write Pros and Cons of Algebraic Specification?

Advantage: Unambiguous and precise

Disadvantage: Difficult to integrate with typical programming language and hard to understand.

21. Define 4GL

- 4GLs (Fourth Generation Languages) are examples of **executable specification languages.**
- 4GLs are successful because there is a lot of commonality across data processing applications.
- 4GLs rely on **software reuse** where common abstractions have been identified and parameterized

22. What is formal technique?

A formal technique is a mathematical method to specify a hardware and/or software system, verify whether a specification is realizable, verify that an implementation satisfies its specification, prove properties of a system without necessarily running the system, etc.

23. List advantages of formal techniques?

- Formal specifications encourage rigor.
- Formal methods usually have a well-founded mathematical basis.
- Formal methods have well-defined semantics.
- The mathematical basis of the formal methods facilitates automating the analysis of specifications
- Formal specifications to obtain immediate feedback

24. List out the basic principles of software project scheduling. (Nov 2017)

There are seven principles of software project scheduling:

- Compartmentalization
- Interdependency

- Time Allocation
- Effort Validation
- Defined Responsibilities
- Defined Outcomes
- Defined Milestones

25. Write the objective of software requirements specification. (Nov 2017)

- fully understand the user requirements
- remove inconsistencies, anomalies, etc. from requirements
- document requirements properly in an SRS document

11 Marks:

1. Explain Software Project Management

- Introduction to Project Planning
- Software Cost Estimation
  - Cost Estimation Models
  - Software Size Metrics
  - Empirical Estimation
  - Heuristic Estimation
  - COCOMO
- Staffing Level Estimation
- Effect of Schedule Compression on Cost
- Summary

Introduction

- Many software projects fail:
  - due to faulty project management practices:
    - It is important to learn different aspects of software project management.
- Goal of software project management:
  - enable a group of engineers to work efficiently towards successful completion of a software project**.**

Responsibility of project managers

- Project proposal writing,
- Project cost estimation,
- Scheduling,
- Project staffing,

- Project monitoring and control,
- Software configuration management,
- Risk management,
- Managerial report writing and presentations, etc.

Introduction

- A project manager's activities are varied.
    - can be broadly classified into:
        - project planning,
        - project monitoring and control activities.

ProjectPlanning

Once a project is found to be feasible, project managers undertake project planning.

Project Planning Activities

- Estimation:
    - Effort, cost, resource, and project duration
- Project scheduling:
- Staff organization:
    - staffing plans
- Risk handling:
    - identification, analysis, and abatement procedures
- Miscellaneous plans:
    - quality assurance plan, configuration management plan, etc.
- Requires utmost care and attention --- commitments to unrealistic time and resource estimates result in:
    - irritating delays.

- customer dissatisfaction
- adverse affect on team morale
  - poor quality work
- project failure.

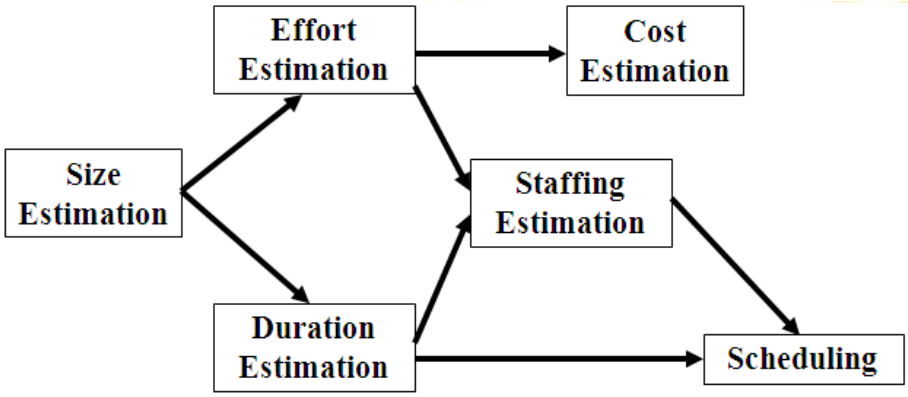
Sliding Window Planning

- Involves project planning over several stages:
  - protects managers from making big commitments too early.
  - More information becomes available as project progresses.
    - Facilitates accurate planning

SPMP Document

- After planning is complete:
  - Document the plans:
  - in a Software Project Management Plan(SPMP) document.
- Organization of SPMP Document
  - Introduction (Objectives, Major Functions, Performance Issues, Management and Technical Constraints)
  - Project Estimates (Historical Data, Estimation Techniques, Effort, Cost, and Project Duration Estimates)
  - Project Resources Plan (People, Hardware and Software, Special Resources)
  - Schedules (Work Breakdown Structure, Task Network, Gantt Chart Representation, PERT Chart Representation)
  - Risk Management Plan (Risk Analysis, Risk Identification, Risk Estimation, Abatement Procedures)
  - Project Tracking and Control Plan
  - Miscellaneous Plans(Process Tailoring, Quality Assurance)

- Software Cost Estimation
- Determine size of the product.
- From the size estimate,
    - Determine the effort needed.
- From the effort estimate,
    - Determine project duration, and cost.



2. Describe Metrics for Project Size Estimations

The project size is a measure of the problem complexity in terms of the effort and time required to develop the product.

Two metrics being used to estimate size:

- Lines of code (LOC)
- Function point (FP)

1) LOC (Lines of Code):

- Simplest and most widely used metric.
- Comments and blank lines should not be counted.

Disadvantages of Using LOC

- Size can vary with coding style.
- Focuses on coding activity alone.
- Correlates poorly with quality and efficiency of code.
- Penalizes higher level programming languages, code reuse, etc.
- Measures lexical/textual complexity only.
    - does not address the issues of structural or logical complexity.
- Difficult to estimate LOC from problem description.
- So not useful for project planning

2) Function Point Metric

- Overcomes some of the shortcomings of the LOC metric
- Proposed by Albrecht in early 80's:
    - FP=4 X#inputs + 5 X #Outputs + 4 X #inquiries + 10 X #files + 10 X #interfaces
- Input:
    - A set of related inputs is counted as one input.
- Output:

        - A set of related outputs is counted as one output.
    - Inquiries:
        - Each user query type is counted.
    - Files:
        - Files are logically related data and thus can be data structures or physical files.
    - Interface:
        - Data transfer to other systems.
    - Suffers from a major drawback:
        - the size of a function is considered to be independent of its complexity.
    - Extend function point metric:
        - **Feature Point metric:**

            It considers an extra parameter Algorithm Complexity.
    - Proponents claim:
        - FP is language independent.
        - Size can be easily derived from problem description

Software Cost Estimation

- Three main approaches to estimation:
    - Empirical
    - Heuristic
    - Analytical
- Software Cost Estimation Techniques

  Empirical techniques:
    - An educated guess based on past experience.

  Heuristic techniques:
    - Assume that the characteristics to be estimated can be expressed in terms of some mathematical expression.

  Analytical techniques:
    - Derive the required results starting from certain simple assumptions.

3. Explain Project Estimation Techniques

Estimation of various project parameter is a basic project planning activity.
The important project parameters that are estimated include: Project size, effort required to develop the software, project duration and cost. These estimates form the basis for resource planning and scheduling.
There are three broad categories of estimation techniques

1. Empirical estimation technique
2. Heuristic technique
3. Analytical estimation technique

1. Empirical Size Estimation Techniques

Empirical estimation techniques are based on making an educated guess of the project parameters. There are two basic empirical estimation techniques:

- Expert Judgement Technique

- Delphi Cost Estimation

Expert Judgement:

- An euphemism for guess made by an expert.
- Suffers from individual bias.

Delphi Estimation:

- overcomes some of the problems of expert judgement.

Expert judgement

- Experts divide a software product into component units:
  - e.g. GUI, database module, data communication module, billing module, etc.
- Add up the guesses for each of the components.

Delphi Estimation:

- Team of Experts and a coordinator.
- Experts carry out estimation independently:
  - mention the rationale behind their estimation.
  - coordinator notes down any extraordinary rationale:
    - circulates among experts.
- Experts re-estimate.
- Experts never meet each other to discuss their viewpoints.

2. <u>Heuristic Estimation Techniques (or) Constructive Cost estimation Model(COCOMO): (NOV 2018)</u>

Heuristic Techniques assume that the relationships among the different project parameters can be modeled using suitable mathematical expressions. Once the basic (independent) parameters are known, the other (dependent) parameters can be easily determined by substituting the value of the basic parameters in the mathematical expression

<u>Single Variable Model:</u>

- Parameter to be Estimated=C1(Estimated Characteristic)^{d1}

<u>Multivariable Model:</u>

- Assumes that the parameter to be estimated depends on more than one characteristic.
- Parameter to be Estimated=C1(Estimated Characteristic)^{d1}+ C2(Estimated Characteristic)^{d2}+…
- Usually more accurate than single variable models.

<u>COCOMO Model</u>

- COCOMO (COnstructiveCOstMOdel) proposed by Boehm.
- Divides software product developments into 3 categories:
  - Organic
  - Semidetached
  - Embedded

COCOMO Product classes

- Roughly correspond to:
  - application, utility and system programs respectively.
    - Data processing and scientific programs are considered to be application programs.
    - Compilers, linkers, editors, etc., are utility programs.
    - Operating systems and real-time system programs, etc. are system programs.

Elaboration of Product classes

- <u>Organic:</u>
  - Relatively small groups
    - working to develop well-understood applications.
- <u>Semidetached:</u>
  - Project team consists of a mixture of experienced and inexperienced staff.

- Embedded:
  - The software is strongly coupled to complex hardware, or real-time systems.
- For each of the three product categories:
  - From size estimation (in KLOC), Boehm provides equations to predict:
    - project duration in months
    - effort in programmer-months
- Boehm obtained these equations:
  - Examined historical data collected from a large number of actual projects.
- Software cost estimation is done through three stages:
  a) Basic COCOMO,
  b) Intermediate COCOMO,
  c) Complete COCOMO.

a) Basic COCOMO

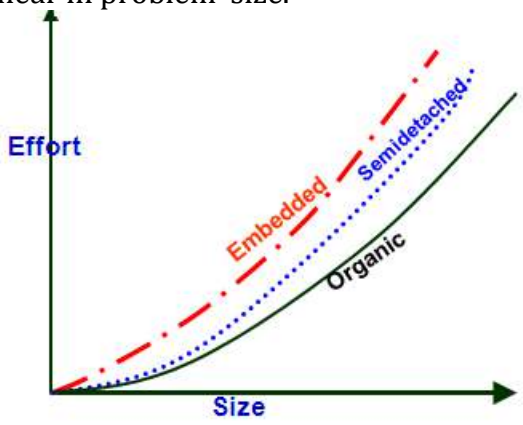
- Gives only an approximate estimation:
  - Effort = a1 (KLOC)^{a2}
  - Tdev = b1 (Effort)^{b2}
    - KLOC is the estimated kilo lines of source code,
    - a1,a2,b1,b2 are constants for different categories of software products,
    - Tdev is the estimated time to develop the software in months,
    - Effort estimation is obtained in terms of person months (PMs).

Development Effort Estimation

- Organic :
  - Effort = 2.4 (KLOC)$^{1.05}$ PM
- Semi-detached:
  - Effort = 3.0(KLOC)$^{1.12}$ PM
- Embedded:
  - Effort = 3.6 (KLOC)$^{1.20}$PM

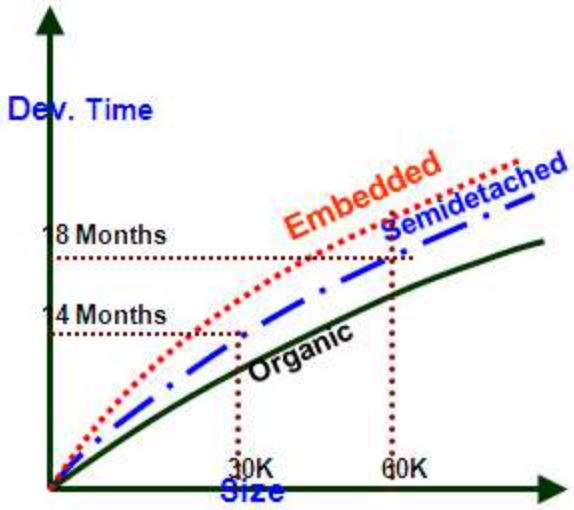
Development Time Estimation

- Organic:
  - Tdev = 2.5 (Effort)$^{0.38}$ Months
- Semi-detached:
  - Tdev = 2.5 (Effort)$^{0.35}$ Months
- Embedded:
  - Tdev = 2.5 (Effort)$^{0.32}$ Months
- Effort is somewhat super-linear in problem size.



- Development time
  - Sublinear function of product size.
- When product size increases two times,
  - Development time does not double.

- Time taken:
  - Almost same for all the three product categories.



- Development time does not increase linearly with product size:
  - For larger products more parallel activities can be identified:
    - can be carried out simultaneously by a number of engineers.
- Development time is roughly the same for all the three categories of products:
  - For example, a 60 KLOC program can be developed in approximately 18 months
    - regardless of whether it is of organic, semi-detached, or embedded type.
  - There is more scope for parallel activities for system and application programs,
    - than utility programs.

Example Assume that the size of an organic type software product has been estimated to be 32,000 lines of source code. Assume that the average salary of software developers is Rs. 15,000 per month. Determine the effort required to develop the software product, the nominal development time, and the cost to develop the product.

From the basic COCOMO estimation formula for organic software:

$\text{Effort} = 2.4 \times (32)^{1.05} = 91 \text{ PM}$

$\text{Nominal development time} = 2.5 \times (91)^{0.38} = 14 \text{ months}$

$\text{Cost required to develop the product} = 91 \times 15,000 = \text{Rs. } 1,465,000$

b) **Intermediate COCOMO**

- Basic COCOMO model assumes
  - Effort and development time depend on product size alone.
- However, several parameters affect effort and development time:
    - Reliability requirements
    - Availability of CASE tools and modern facilities to the developers
    - Size of data to be handled
- For accurate estimation,
  - the effect of all relevant parameters must be considered:
  - Intermediate COCOMO model recognizes this fact:
    - Refines the initial estimate obtained by the basic COCOMO by using a set of 15 cost drivers (multipliers).
- If modern programming practices are used,
  - Initial estimates are scaled downwards.
- If there are stringent reliability requirements on the product :

- Initial estimate is scaled upwards.
- Rate different parameters on a scale of one to three:
  - Depending on these ratings,
    - Multiply cost driver values with the estimate obtained using the basic COCOMO.
- Cost driver classes:
  - Product: Inherent complexity of the product, reliability requirements of the product, etc.
  - Computer: Execution time, storage requirements, etc.
  - Personnel: Experience of personnel, etc.
  - Development Environment: Sophistication of the tools used for software development.

Shortcoming of basic and intermediate COCOMO models

- Both models:consider a software product as a single homogeneous entity:
  - However, most large systems are made up of several smaller sub-systems.
    - Some sub-systems may be considered as organic type, some may be considered embedded, etc.
    - for some the reliability requirements may be high, and so on.

c) Complete COCOMO

- Cost of each sub-system is estimated separately.
- Costs of the sub-systems are added to obtain total cost.
- Reduces the margin of error in the final estimate.
- Complete COCOMO Example

  A Management Information System (MIS) for an organization having offices at several places across the country:
  - Database part (semi-detached)
  - Graphical User Interface (GUI) part (organic)
  - Communication part (embedded)
- Costs of the components are estimated separately:
  - Summed up to give the overall cost of the system.

3) Halstead's Software Science (or) Analytical estimation technique:

An analytical technique to estimate:

a. size,
b. development effort,
c. Development time.

Halstead used a few primitive program parameters

- number of operators and operands

Derived expressions for:

- over all program length,
- potential minimum volume
- actual volume,
- language level,
- effort, and
- Development time.

For a give program, let:

- η_1 be the number of unique operators used in the program
- $\eta_2$be the number of unique operands used in the program
- N_1 be the total number of operators used in the program
- N_2be the total number of operands used in the program

Program length $N=N1+ N2$,

Program vocabulary $\eta= \eta_1+\eta_2$

Program volume $V = N \log_2\eta$

Potential minimum volume $V^*=(2+\eta_2)\log_2(2+\eta_2)$
Programming level L= V*/V
Programming effort E=V/L
Programmer's time T=E/S, where S is the speed of mental s(recommended value for programming applications is 18)
Program length estimation $N= \eta_1\log_2\eta_1+\eta_2\log_2\eta_2$

Example Let us consider the following C program:

```
main()
{
  int a,b,c,avg;

  scanf("%d %d %d",&a,&b,&c);
  avg=(a+b+c)/3;
  printf("avg= %d",avg);
}
```

The unique operators are: `main, (), {}, int, scanf, &, ",", ";", =, +, /, printf`
The unique operands are: `a,b,c,&a,&b,&c,a+b+c,avg,3,"%d %d %d", "avg=%d"`
Therefore,

$$\eta_1 = 12, \eta_2 = 11$$
$$\text{Estimated Length} = (12 * \log 12 + 11 * \log 11)$$
$$= (12 * 3.58 + 11 * 3.45) = (43 + 38) = 81$$
$$\text{Volume} = \text{Length} * \log(23) = 81 * 4.52 = 366$$

Staffing Level Estimation

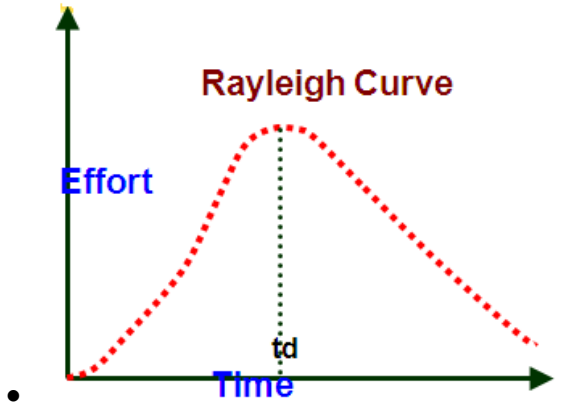
- Number of personnel required during any development project is not constant.

Norden's Work

Norden in 1958 analyzed many R&D projects, and observed: Rayleigh curve represents the number of full-time personnel required at any time.

Rayleigh Curve

- Rayleigh curve is specified by two parameters:
  - td the time at which the curve reaches its maximum
  - K the total area under the curve.
- L=f(K, td)



$$E = \frac{K}{{t_d}^2} * t * e^{\frac{-t^2}{2{t_d}^2}}$$

Where E is the effort required at time t.

Putnam's Work:

- In 1976, Putnam studied the problem of staffing of software projects:
- Observed that the level of effort required in software development efforts has a similar envelope.
- Found that the Rayleigh-Norden curve relates the number of delivered lines of code to effort and development time.
    - Putnam analysed a large number of army projects, and derived the expression:

$$L = C_k K^{1/3} t_d^{\frac{4}{3}}$$

        - K is the effort expended and L is the size in KLOC.
        - td is the time to develop the software.
        - C_k is the state of technology constant reflects factors that affect programmer productivity.
        - Ck=2 for poor development environment
        - no methodology, poor documentation, and review, etc.
        - Ck=8 for good software development environment
        - software engineering principles used
            - Ck=11 for an excellent environment

Rayleigh Curve

- Very small number of engineers are needed at the beginning of a project carry out planning and specification.
- As the project progresses: more detailed work is required,
    - Number of engineers slowly increases and reaches a peak.
- Putnam observed that:
  the time at which the Rayleigh curve reaches its maximum value corresponds to system testing and product release.
  After system testing, the number of project staff falls till product installation and delivery.
- From the Rayleigh curve observe that:
  Approximately 40% of the area under the Rayleigh curve is to the left of td and 60% to the right.

Effect of Schedule Change on Cost

By using the Putnam's proposed expression for L,

$$K = L^3/(C_k^3 t_d^4)$$

or

$$K = C/{t_d}^4$$

For the same product size, $C = \frac{L^3}{C_k^3}$ is a constant.
or

$$K_1/K_2 = t_{d_2}^4/t_{d_1}^4$$

Observe:
a relatively small compression in delivery schedule can result in substantial penalty on human effort.

Also, observe: benefits can be gained by using fewer people over a somewhat longer time span.

Example

If the estimated development time is 1 year, then in order to develop the product in 6 months, the total effort and hence the cost increases 16 times.

In other words, the relationship between effort and the chronological delivery time is highly nonlinear.

- Putnam model indicates extreme penalty for schedule compression and extreme reward for expanding the schedule.
- Putnam estimation model works **reasonably well for very large systems**,
  But seriously overestimates the effort for medium and small systems.
- Boehm observed:
  "There is a limit beyond which the schedule of a software project cannot be reduced by buying any more personnel or equipment."
  This limit occurs roughly at 75% of the nominal time estimate.
- If a project manager accepts a customer demand to compress the development time by more than 25% very unlikely to succeed.
  - every project has only a limited amount of parallel activities
  - Sequential activities cannot be speeded up by hiring any number of additional engineers.
  - Many engineers have to sit idle.

Jensen Model

- Jensen model is very similar to Putnam model.
- attempts to soften the effect of schedule compression on effort
- makes it applicable to smaller and medium sized projects.
- Jensen proposed the equation:

$$L = C_{te} t_d K^{1/2}$$

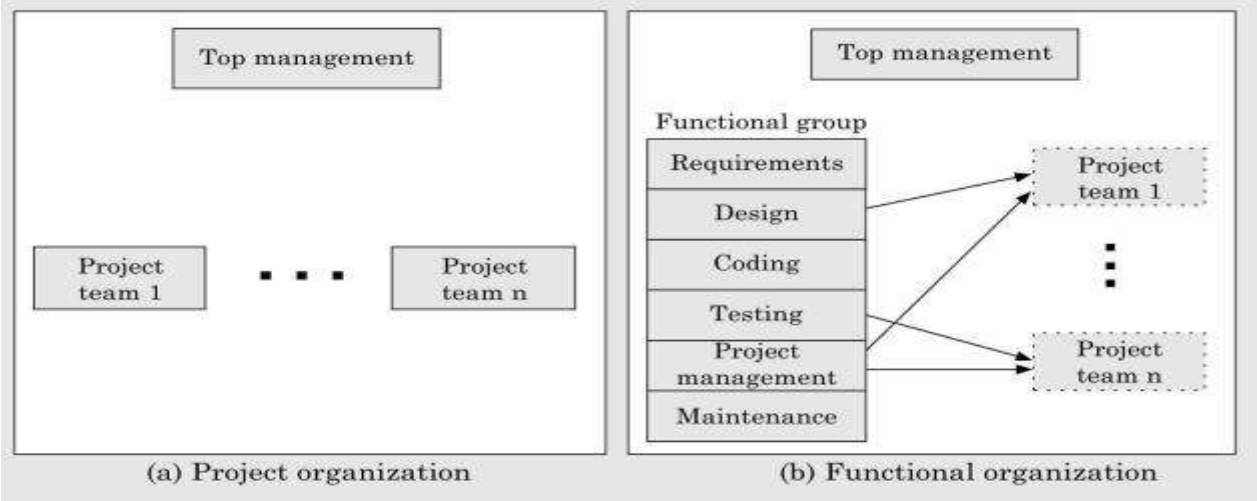
Where,

- C_{te} is the effective technology constant,
- t_d is the time to develop the software, and
- K is the effort needed to develop the software.

4. Explain Organization and Team Structure

Organization Structure

1. Functional organization
2. Project organization



1. Functional Organization:

- Engineers are organized into functional groups, e.g.
  - specification, design, coding, testing, maintenance, etc.
- Engineers from functional groups get assigned to different projects

Advantages of Functional Organization

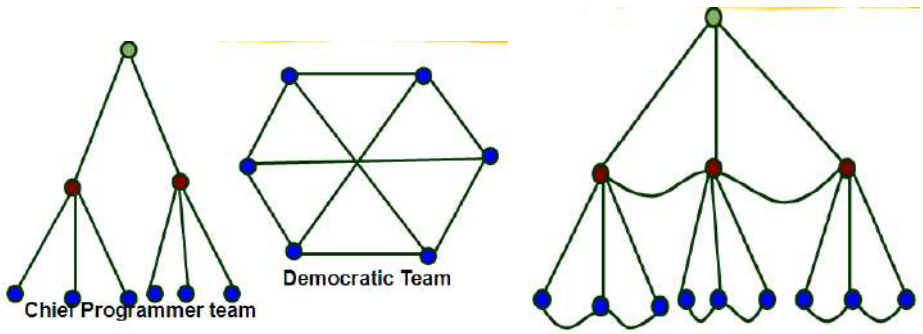
- Specialization
- Ease of staffing
- Good documentation is produced
  - different phases are carried out by different teams of engineers.
- Helps identify errors earlier.

2. Project Organization

- Engineers get assigned to a project for the entire duration of the project
  - Same set of engineers carry out all the phases
- Advantages:
  - Engineers save time on learning details of every project.
  - Leads to job rotation

Team Structure

- Problems of different complexities and sizes require different team structures:
  - Chief programmer: suitable for routine work.
  - Democratic: Small teams doing R&D type work
  - Mixed: Large projects



Mixed organization

Democratic Teams

- Suitable for:
  - small projects requiring less than five or six engineers
  - research-oriented projects
- A manager provides administrative leadership:

- at different times different members of the group provide technical leadership.

- Democratic organization provides
  - higher morale and job satisfaction to the engineers
  - therefore leads to less employee turnover.
- Suitable for less understood problems,
  - a group of engineers can invent better solutions than a single individual.

Disadvantage:

- team members may waste a lot time arguing about trivial points:
  - absence of any authority in the team.

Chief Programmer Team

- A senior engineer provides technical leadership:
  - partitions the task among the team members.
  - Verifies and integrates the products developed by the members.
- Works well when
  - the task is well understood also within the intellectual grasp of a single individual,
  - importance of early completion outweighs other factors
    - team morale, personal development, etc.
- Chief programmer team is subject to single point failure:
  - too much responsibility and authority is assigned to the chief programmer.

Mixed Control Team Organization

- Draws upon ideas from both:
  - democratic organization and
  - Chief-programmer team organization.
- Communication is limited
  - to a small group that is most likely to benefit from it.
- Suitable for large organizations.
-

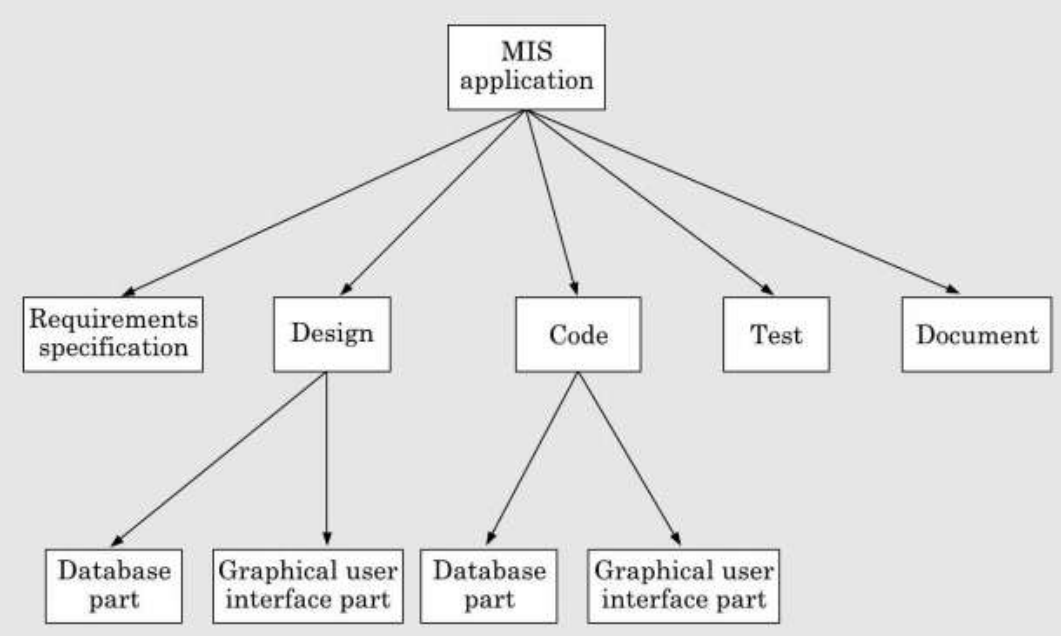
Project scheduling

Project-task scheduling is an important project planning activity. It involves deciding which tasks would be taken up when. In order to schedule the project activities, a software project manager needs to do the following:

1. Identify all the tasks needed to complete the project.
2. Break down large tasks into small activities.
3. Determine the dependency among different activities.
4. Establish the most likely estimates for the time durations necessary to complete the activities.
5. Allocate resources to activities.
6. Plan the starting and ending dates for various activities.
7. Determine the critical path. A critical path is the chain of activities that determines the duration of the project.

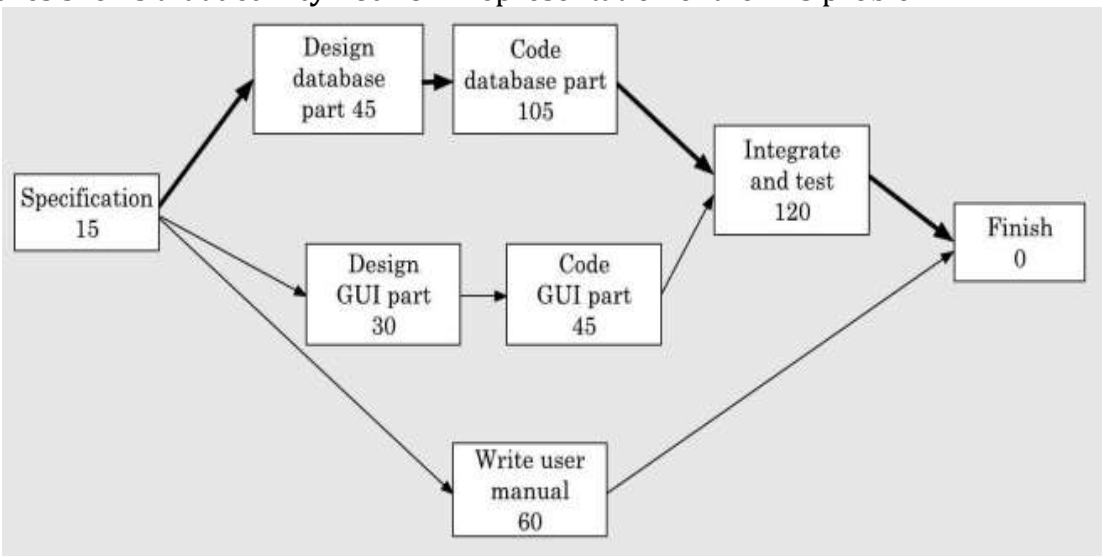
Work breakdown structure:

- Work Breakdown Structure (WBS) is used to decompose a given task set recursively into small activities. WBS provides a notation for representing the major tasks need to be carried out in order to solve a problem.
- Figure shows that work breakdown structure of an MIS problem.



Activity networks and critical path method:-

- WBS representation of a project is transformed into an activity network by representing activities identified in WBS along with their interdependencies.
- An activity network shows the different activities making up a project, their estimated durations, and interdependencies
- Each **activity** is represented by a rectangular node and the **duration** of the activity is shown alongside each task.
- Figures shows that **activity network** representation of the MIS problem



Critical Path Method (CPM)

From the activity network representation following analysis can be made.

- The minimum time (MT) to complete the project is the maximum of all paths from start to finish.
- The earliest start (ES) time of a task is the maximum of all paths from the start to the task. The latest start time is the difference between MT and the maximum of all paths from this task to the finish.
- The earliest finish time (EF) of a task is the sum of the earliest start time of the task and the duration of the task.

- The latest finish (LF) time of a task can be obtained by subtracting maximum of all paths from this task to finish from MT.
- The slack time (ST) is LS – EF and equivalently can be written as LF – EF. The slack time (or float time) is the total time that a task may be delayed before it will affect the end time of the project.

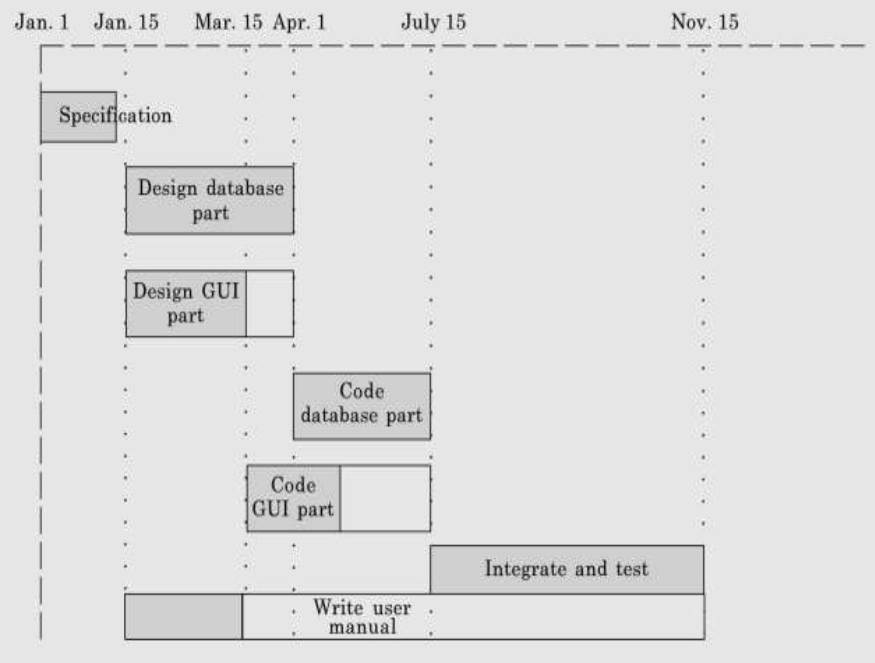
The slack time indicates the "flexibility" in starting and completion of tasks. **A critical task is one with a zero slack time**. A path from the start node to the finish node containing only critical tasks is called a **critical path**. These parameters for different tasks for the MIS problem are shown in the following table.

Task	*ES*	*EF*	*LS*	*LF*	*ST*
Specification	0	15	0	15	0
Design database	15	60	15	60	0
Design GUI part	15	45	90	120	75
Code data base	60	165	60	165	0
Code GUI part	45	90	120	165	75
Integrate and test	165	285	165	285	0
Write user manual	15	75	225	285	210

- The critical paths are all the paths whose duration equals MT. The critical path in figure activity network is shown with a thick arrow.

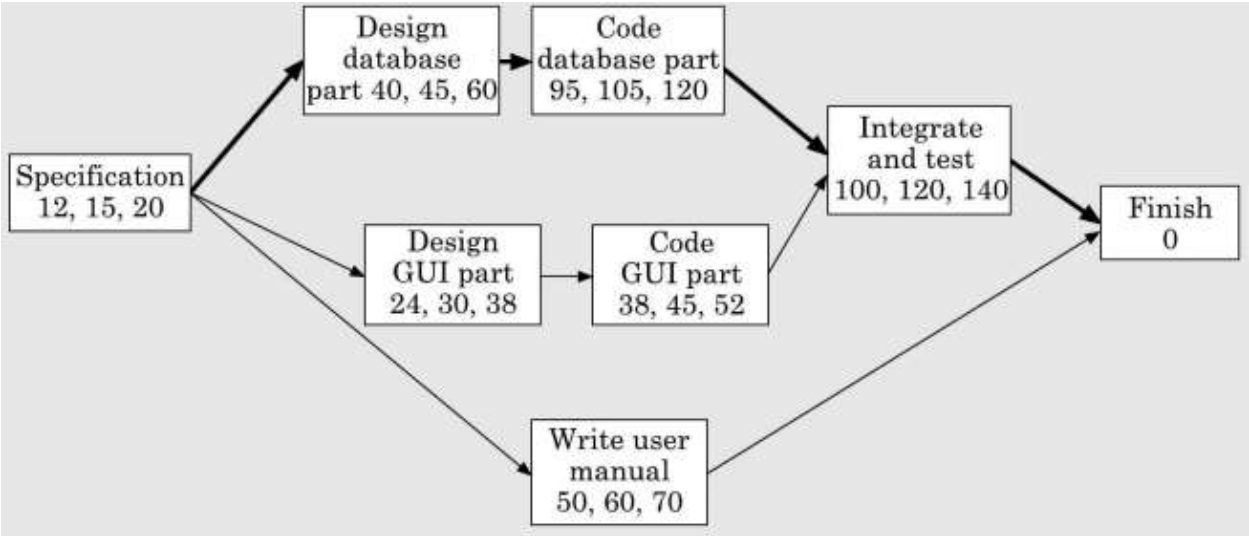
Gantt chart:-

- Gantt charts are mainly used to **allocate resources to activities**.
- The resources allocated to activities include staff, hardware, and software. Gantt charts (named after its developer Henry Gantt) are useful for resource planning.
- A Gantt chart is a **special type of bar chart** where each bar represents an activity. The bars are drawn along a time line. The **length of each bar is proportional to the duration of time planned** for the corresponding activity.
- A Gantt chart representation of MIS problem shown below.



PERT chart:-

- PERT (**Project Evaluation and Review Technique**) charts consist of a network of boxes and arrows. The boxes represent activities and the arrows represent task dependencies
- The boxes of PERT charts are usually annotated with the **pessimistic, likely, and optimistic estimates** for every task.



- Since all possible completion times between the minimum and maximum duration for every task has to be considered, there are not one but many critical paths, depending on the permutations of the estimates for each task.
- Gantt chart representation of a project schedule is helpful in planning the utilization of resources, while PERT chart is useful for monitoring the timely progress of activities.

Project Monitoring and Control:-

- Once the project gets underway, the project manager has to monitor the project continuously to ensure that it is progressing as per plan.**PERT chart are useful** for project monitoring and control.
- The project manager designates certain key events such as **completion of some important activity as milestones.**
- A **critical path** in graph is a path along which every milestone is critical to meeting the project deadline.

5. Describe Risk Management

- A risk is any anticipated unfavourable event or circumstance that can occur while a project is underway.
- Risk management aims at reducing the impact of all kinds of risks that might affect a project.
- Risk management consists of three essential activities:
  - Risk identification
  - Risk assessment
  - Risk containment

Risk Identification:-

- Risks in the project must be minimized by making effective risk management plans.
- Risks that are likely to affect a project must be identified and listed.

- Types:
  - Project risks
  - Technical risks
  - Business risks
- **Project risks**
  Project risks concern varies forms of budgetary, schedule, personnel, resource, and customer-related problems. An important project risk is schedule slippage.
- **Technical risks**
  Technical risks concern potential design, implementation, interfacing, testing, and maintenance problems.
- **Business risks.**
  This type of risks include risks of building an excellent product that no one wants, losing budgetary or personnel commitments, etc.

Risk assessment:-

- The objective of risk assessment is to rank the risks in terms of their damage causing potential.
- For risk assessment, first each risk should be rated in two ways:
  - The likelihood of a risk coming true (denoted as r).
  - The consequence of the problems associated with that risk (denoted as s).
- Based on these two factors, the priority of each risk can be computed:

$$p = r * s$$

Where, p is the **priority** with which the risk must be handled,
r is the probability ofthe **risk becoming true**, and
s is the **severity of damage caused** due to the risk becoming true.

Risk containment:-

After all the identified risks of a project are assessed, plans must be made to contain the most damaging and the most likely risks.
There are three main strategies to plan for risk containment:

Avoid the risk: This may take several forms such as discussing with the customer to change the requirements to reduce the scope of the work, giving incentives to the engineers to avoid the risk of manpower turnover, etc.

Transfer the risk: This strategy involves getting the risky component developed by a third party, buying insurance cover, etc.

Risk reduction: This involves planning ways to contain the damage due to a risk. For example, if there is risk that some key personnel might leave, new recruitment may be planned.

Risk leverage

To choose between the different strategies of handling a risk, the project manager must consider the cost of handling the risk and the corresponding reduction of risk. For this the risk leverage of the different risks can be computed.

Risk leverage is the difference in risk exposure divided by the cost of reducing the risk. More formally,

$$\text{Risk leverage} = \frac{\text{Risk exposure before reduction} - \text{Risk exposure after reduction}}{\text{Cost of reduction}}$$

6. Explain Software configuration management (or) Discuss in detail about software configuration techniques. (Nov 2017)

- Software configuration management deals with effectively tracking and controlling the configuration of a software product during its life cycle.
- The state of each deliverable object changes as development progresses and also as bugs are detected and fixed.

Release vs. Version vs. Revision

- A new version of a software is created when there is **a significant change in functionality, technology, or the hardware it runs on,** etc.
- On the other hand a new revision of a software refers to **minor bug fix in that software**.
- A new release is created if there is only **a bug fix, minor enhancements to the functionality, usability, etc.**

Necessity of software configuration management

The following are some of the important problems that appear if configuration management is not used.

- Inconsistency problem when the objects are replicated.
- Problems associated with concurrent access.
- Providing a stable development environment
- System accounting and maintaining status information. System accounting keeps
  - track of who made a particular change and when the change was made.
- Handling variants**.** Existence of variants of a software product causes some peculiar problems.

Software configuration management activities

Configuration management is carried out through two principal activities:

- **Configuration identification** involves deciding which parts of the system should be kept track of.
- **Configuration control** ensures that changes to a system happen smoothly.

Configuration identification

- The project manager normally classifies the objects associated with a software development effort into three main categories: **controlled, precontrolled, and uncontrolled.**
- Controlled objects are those which are already **put under configuration control**. One must follow some formal procedures to change them.

- Precontrolled objects are **not yet under configuration control**, but will eventually be under configuration control.
- Uncontrolled objects are not and will not be subjected to configuration control.

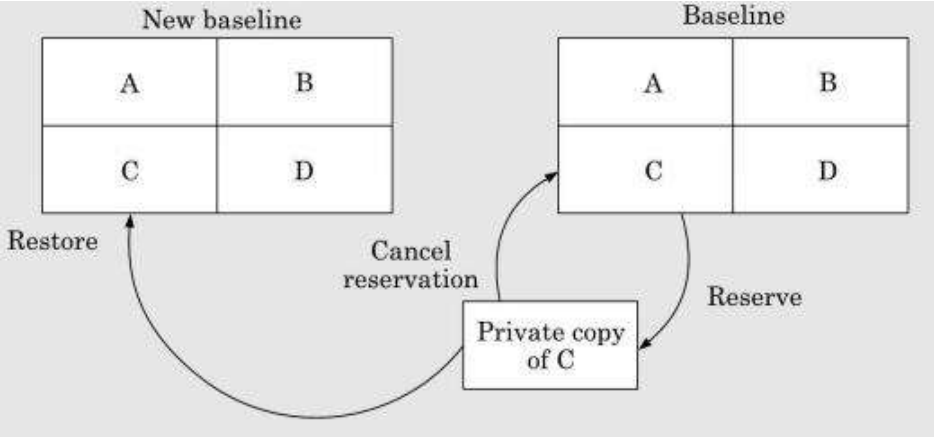
Controllable objects include both controlled and precontrolled objects. Typical controllable objects include:

- Requirements specification document
- Design documents
- Tools used to build the system, such as compilers, linkers, lexical analyzers, parsers, etc.

- Source code for each module
- Test cases
- Problem reports

Configuration control:-

- Configuration control allows only authorized changes to the controlled objects to occur and prevents unauthorized changes
- In order to change a controlled object such as a module, a developer can **get a private copy of the module by a reserve operation** as shown in fig. 12.5.
- Configuration management tools **allow only one person to reserve a module at a time.**
- Once an object is reserved, it **does not allow any one else to reserve this module until the reserved module is restored** as shown in fig. Thus, by preventing more than one engineer to simultaneously reserve a module, the problems associated with concurrent access are solved.



Change control board (CCB):

- The CCB is usually constituted from among the **development team members**.
- For every change that needs to be carried out, the CCB **reviews the changes made to the controlled object and certifies several things about the change**s follows:
  - ✓ Change is well-motivated.
  - ✓ Developer has considered and documented the effects of the change.
  - ✓ Changes interact well with the changes made by other developers.
  - ✓ Appropriate people (CCB) have validated the change, e.g. someone has tested the changed code, and has verified that the change is consistent with the requirement.

7. Explain Formal System Specification

Formal technique

A formal technique is a mathematical method to specify a hardware and/or software system, verify whether a specification is realizable, verify that an implementation satisfies its specification, prove properties of a system without necessarily running the system, etc. The mathematical basis of a formal method is provided by the specification language

Formal specification language

A formal specification language consists of two sets **syn** and **sem,** and a relation **sat** between them

- **Syntactic Domains**

  It consists of alphabet of symbols and set of formation rules to construct well-formed formulas from the alphabet.

- **Semantic Domains**

  Abstract data type specification languages are used to specify algebras, theories, and programs
- **Satisfaction Relation**

  Determine whether an element of the semantic domain satisfies the specifications

Model-oriented vs. property-oriented approaches

In a model-oriented style, one defines a system's behavior **directly by constructing a model of the system** in terms of mathematical structures such as tuples, relations, functions, sets, sequences, etc.

In the property-oriented style, **the system's behavior is defined indirectly** by stating its properties, usually in the form of a set of axioms that the system must satisfy.

Operational semantics

a formal method is the way computations are represented.

- **Linear Semantics:-**

  A run of a system is described by a sequence of events or states.
- **Branching Semantics:-**

  The behavior of a system is represented by a directed graph
- **Maximally parallel semantics:-**

  All the concurrent actions enabled at any state are assumed to be taken together.
- **Partial order semantics:-**

  The semantics ascribed to a system is a structure of states satisfying a partial order relation among the states (events).

Merits of formal requirements specification

- Formal specifications encourage rigor.
- Formal methods usually have a well-founded mathematical basis.
- Formal methods have well-defined semantics.
- The mathematical basis of the formal methods facilitates automating the analysis of specifications
- Formal specifications to obtain immediate feedback

8. Explain Axiomatic Specification

First-order logic is used to write the pre and post-conditions to specify the operations of the system in the form of axioms. The **pre-conditions** capture the requirements on the input parameters of a function. The **post-conditions** are the conditions that must be satisfied when a function completes execution for the function to be considered to have executed successfully.

Steps to develop the axiomatic specifications of a function:

- Establish the range of input values over which the function should behave correctly. Also find out other constraints on the input parameters and write it in the form of a predicate.
- Specify a predicate defining the conditions which must hold on the output of the function if it behaved properly.
- Establish the changes made to the function's input parameters after execution of the function.
- Combine all of the above into pre and post conditions of the function.

Example : Specify the pre- and post-conditions of a function that takes a real number as argument and returns half the input value if the input is less than or equal to 100, or else returns double the value.

$$f(x : real) : real$$

$$pre : x \in R$$

$$post : \{(x \leq 100) \wedge (f(x) = x/2)\} \vee \{(x > 100) \wedge (f(x) = 2 * x)\}$$

Example : Axiomatically specify a function named search which takes an integer array and an integer key value as its arguments and returns the index in the array where the key value is present.

$$search(X : intArray, key : integer) : integer$$

$$pre : \exists i \in [Xfirst...Xlast], X[i] = key$$

$$post : \{(X'[search(X, key)] = key) \wedge (X = X')\}$$

9. Explain Algebraic Specification?

Algebraic specification

An object class or type is specified in terms of relationships existing between the operations defined on that type.

Representation of algebraic specification

Algebraic specifications define a system as a heterogeneous algebra. A heterogeneous algebra is a collection of different sets on which several operations are defined. homogeneous algebras are traditional.

A homogeneous algebra consists of a single set and several operations; {I, +, -, *, /}. In contrast, alphabetic strings together with operations of concatenation and length {A, I, con, len}, is not a homogeneous algebra, since the range of the length operation is the set of integers.

Each set of symbols in the algebra, is called a *sort* of the algebra. To define a heterogeneous algebra, we first need to specify its signature, the involved operations, and their domains and ranges. Using

algebraic specification, we define the meaning of a set of interface procedure by using equations. An algebraic specification is usually presented in four sections.

Types section:-In this section, the sorts (the data types) being used is specified.

Exceptions section:-

This section gives the names of the exceptional conditions that might occur when different operations are carried out. These exception conditions are used in the later sections of an algebraic specification.

For example, in a queue, possible exceptions are no value (empty queue), underflow (removal from an empty queue), overflow etc.

Syntax section:-

This section defines the signatures of the interface procedures. The collection of sets that form input domain of an operator and the sort where the output is produced are called the signature of the operator.

For example, the append operation takes a queue and an element and returns a new queue. It is represented as:

append: queue x element → queue

Equations section:-

This section gives a set of rewrite rules (or equations) defining the meaning of the interface procedures in terms of each other. In general, this section is allowed to contain conditional expressions.

For example, a rewrite rule to identify an empty queue may be written as:

isempty(create()) = true

Example 1:-

Let us specify a FIFO queue supporting the operations *create, append, remove, first,* and *isempty*where the operations have their usual meaning.

Types:

Defines queue uses Boolean, integer

Exceptions: underflow, no value

Syntax:

1. create : φ *→ queue*

2. append : queue x element → queue

3. remove : queue → queue + {underflow}

4. first : queue → element + {no value}

5. isempty : queue → Boolean

Equations:

1. isempty(create()) = true

2. isempty((append(q,e)) = false

3. first(create()) = no value

4. first(append(q,e)) = is isempty(q) then e else first(q)

5. remove(create()) = underflow

6. remove(append(q,e)) = if isempty(q) then create()

else append(remove(q),e)

In this example, there are two basic constructors (*create* and *append*), one extra construction operator (*remove*) and two basic inspectors (*first* and *empty*). Therefore, there are 2 x (1+2) + 0 = 6 equations.

Example 2:-

Let us specify a data type point supporting the operations create, xcoord, ycoord, isequal; where the operations have their usual meaning.

Types:

defines point, uses boolean, integer

Syntax:

1. create : integer × integer → point

2. xcoord : point → integer

3. ycoord : point → integer

4. isequal : point × point → Boolean

Equations:

1. xcoord(create(x, y)) = x

2. ycoord(create(x, y)) = y

3. isequal(create(x1, y1), create(x2, y2)) = ((x1 = x2) and (y1 = y2))

In this example, there is only one basic constructor (create), and three basic inspectors (xcoord, ycoord, and isequal). Therefore, there are only 3 equations.

Properties of algebraic specifications

- Completeness
- Finite termination property
- Unique termination property

10. Explain Requirements Analysis and Specification

- Many projects fail:
  - because they start implementing the system:
  - without determining whether they are building what the customer really wants.
- It is important to learn:
  - Requirements analysis and specification techniques thoroughly.
- **Goals of requirements analysis and specification phase**:
  - fully understand the user requirements
  - remove inconsistencies, anomalies, etc. from requirements
  - document requirements properly in an SRS document
- Consists of **two distinct activities**:
    - Requirements Gathering and Analysis
    - Specification
- The person who undertakes requirements analysis and specification:
  - known as **systems analyst:**
  - collects data pertaining to the product
  - analyzes collected data:
    - to understand what exactly needs to be done.
  - writes the Software Requirements Specification (SRS) document.
- Final output of this phase:
  - Software Requirements Specification **(SRS) Document**.
- The SRS document is reviewed by the customer.
  - Reviewed SRS document forms the basis of all future development activities.

Requirements Analysis

- Requirements analysis consists of **two main activities:**
  - **Requirements gathering**
  - **Analysis of the gathered requirements**
- Analyst gathers requirements through:
  - observation of existing systems,
  - studying existing procedures,

- discussion with the customer and end-users,
- analysis of what needs to be done, etc.

Requirements Gathering

- If the project is to automate some existing procedures
  - e.g., automating existing manual accounting activities,
  - the task of the system analyst is a little easier
  - analyst can immediately obtain:
    - input and output formats
    - accurate details of the operational procedures
- In the absence of a working system,
  - lot of imagination and creativity are required.
- Interacting with the customer to gather relevant data:
  - requires a lot of experience.
- Some desirable attributes of a good system analyst:
  - Good interaction skills,
  - imagination and creativity,
  - experience.

Analysis of the Gathered Requirements

- After gathering all the requirements:
  - analyze it:
    - Clearly understand the user requirements,
    - Detect inconsistencies, ambiguities, and incompleteness.
    - Incompleteness and inconsistencies:
      - resolved through further discussions with the end-users and the customers.

Inconsistent requirement

- Some part of the requirement:
  - contradicts with some other part.
- Example:
  - One customer says turn off heater and open water shower when temperature > 100 C

- Another customer says turn off heater and turn ON cooler when temperature > 100 C

Incomplete requirement

- Some requirements have been omitted:
  - due to oversight.
- Example:
  - The analyst has not recorded:
    when temperature falls below 90 C
    - heater should be turned ON
    - water shower turned OFF.
- Requirements analysis involves:
  - obtaining a clear, in-depth understanding of the product to be developed,
  - remove all ambiguities and inconsistencies from the initial customer perception of the problem.
- It is quite difficult to obtain:
  - a clear, in-depth understanding of the problem:
    - especially if there is no working model of the problem.
- Experienced analysts take considerable time:
  - to understand the exact requirements the customer has in his mind.
- Experienced systems analysts know -
  often as a result of painful experiences ---
  - without a clear understanding of the problem, it is impossible to develop a satisfactory system.
- Several things about the project should be clearly understood by the analyst:
  - What is the problem?
  - Why is it important to solve the problem?
  - What are the possible solutions to the problem?
  - What complexities might arise while solving the problem?
- Some anomalies and inconsistencies can be very subtle:
  - escape even most experienced eyes.
  - If a formal model of the system is constructed,
    - many of the subtle anomalies and inconsistencies get detected.

- After collecting all data regarding the system to be developed,
    - remove all inconsistencies and anomalies from the requirements,
    - systematically organize requirements into a Software Requirements Specification (SRS) document.

Software Requirements Specification (NOV 2018)

- Main aim of requirements specification:
    - systematically organize the requirements arrived during requirements analysis
    - document requirements properly.
- The **SRS document** is useful in various contexts:
    - **statement of user needs**
    - **contract document**
    - **reference document**
    - **definition for implementation**

Software Requirements Specification: A Contract Document

- Requirements document is a reference document.
- SRS document is a contract between the development team and the customer.
    - Once the SRS document is approved by the customer,
        - any subsequent controversies are settled by referring the SRS document.
- Once customer agrees to the SRS document:
    - development team starts to develop the product according to the requirements recorded in the SRS document.
- The final product will be acceptable to the customer:
    - as long as it satisfies all the requirements recorded in the SRS document.
- The SRS document is known as black-box specification:
    - the system is considered as a black box whose internal details are not known.
    - only its visible external (i.e. input/output) behaviour is documented.

- SRS document concentrates on:

- what needs to be done
- carefully avoids the solution ("how to do") aspects.

- The SRS document serves as a contract
  - between development team and the customer.
  - Should be carefully written
- The requirements at this stage:
  - written using end-user terminology.
- If necessary:
  - later a formal requirement specification may be developed from it.

SRS document, normally contains **three important parts**:

- **functional requirements,**
- **non-functional requirements,**
- **Constraints on the system.**

Functional requirements:

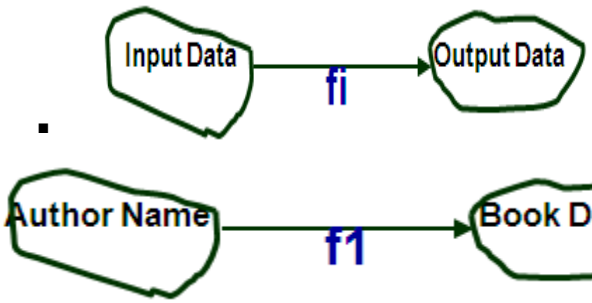
- Functional requirements describe:
  - A set of high-level requirements
  - Each high-level requirement:
    - takes in some data from the user
    - outputs some data to the user
  - Each high-level requirement:
    - might consist of a **set of identifiable functions**
- For each high-level requirement:
  - every function is described in terms of
    - input data set
    - output data set

      Processing required to obtain the output data set from the input data set

Example: Functional Requirement

- F1: Search Book
  - Input:

        - an author's name:
    - Output:
        - details of the author's books and the locations of these books in the library.



Non-functional Requirements

- Characteristics of the system which cannot be expressed as functions:
    - maintainability,
    - portability,
    - usability, etc.
- Non-functional requirements include:
    - reliability issues,
    - performance issues,
    - human-computer interface issues,
    - Interface with other external systems,
    - security, maintainability, etc.

Constraints

- Constraints describe things that the system should or should not do.
    - For example,
        - standards compliance
        - how fast the system can produce results
            - so that it does not overload another system to which it supplies data, etc.
- Examples of constraints
- Hardware to be used,
- Operating system
    - or DBMS to be used

- Capabilities of I/O devices
- Standards compliance
- Data representations
    - by the interfaced system

Organization of the SRS Document

- Introduction.
- Functional Requirements
- Non-functional Requirements
    - External interface requirements
    - Performance requirements
- Constraints
- Example Functional Requirements
- List all functional requirements
    - with proper numbering.
- Req. 1:
    - Once the user selects the "search" option,
        - he is asked to enter the key words.
    - The system should output details of all books
        - whose title or author name matches any of the key words entered?
        - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Catalogue Number, Location in the Library.
- Req. 2:
    - When the "renew" option is selected,
        - the user is asked to enter his membership number and password.
    - After password validation,
        - the list of the books borrowed by him are displayed.
    - The user can renew any of the books:
        - by clicking in the corresponding renew box.
- Req. 1:
- R.1.1:

- Input: “search” option,
- Output: user prompted to enter the key words.

- R1.2:
  - Input: key words
  - Output: Details of all books whose title or author name matches any of the key words.
    - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Catalog Number, and Location in the Library.
  - Processing: Search the book list for the keywords

- Req. 2:
- R2.1:
  - Input: “renew” option selected,
  - Output: user prompted to enter his membership number and password.
- R2.2:
  - Input: membership number and password
  - Output:
    - list of the books borrowed by user are displayed. User prompted to enter books to be renewed or
    - user informed about bad password
  - Processing: Password validation, search books issued to the user from borrower list and display.
- R2.3:
  - Input: user choice for renewal of the books issued to him through mouse clicks in the corresponding renew box.
  - Output: Confirmation of the books renewed
  - Processing: Renew the books selected by the in the borrower list.

Properties of a good SRS document

- It should be **concise**
  - and at the same time should not be ambiguous.
- It should specify what the system must do
  - and not say how to do it.
- **Easy to change**.,
  - i.e. it should be well-structured.
- It should be **consistent**.

- It should be **complete.**
- It should be **traceable**
  - you should be able to trace which part of the specification corresponds to which part of the design and code, etc and vice versa.
- It should be **verifiable**
  - e.g. "system should be user friendly" is not verifiable

Examples of Bad SRS Documents

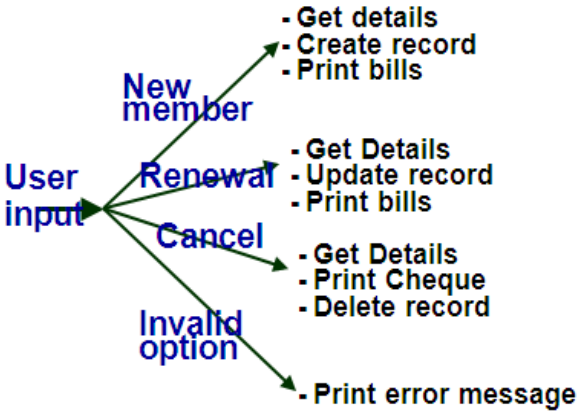
- Unstructured Specifications:
- Noise: Presence of text containing information irrelevant to the problem.
- Silence:aspects important to proper solution of the problem are omitted.
- Overspecification:Addressing "how to" aspects
- For example, "Library member names should be stored in a sorted descending order" Overspecification restricts the solution space for the designer.
- Contradictions:Contradictions might arise if the same thing described at several places in different ways.
- Ambiguity:Literary expressionsUnquantifiable aspects, e.g. "good user interface"
- Forward References:References to aspects of problemdefined only later on in the text.
- Wishful Thinking:Descriptions of aspects for which realistic solutions will be hard to find.

Representation of complex processing logic:

- Decision trees
- Decision tables
- Decision Trees
- Decision trees:
  - edges of a decision tree represent conditions
  - leaf nodes represent actions to be performed.
- A decision tree gives a graphic view of:
  - logic involved in decision making
  - corresponding actions taken.
- Example: LMS
- A Library Membership automation Software (LMS) should support the following three options:
  - new member,
  - renewal,
  - Cancel membership.
- When the new member option is selected,
  - the software asks details about the member:
    - name,

- address,
- Phone number, etc.

- If proper information is entered,
  - a membership record for the member is created
  - a bill is printed for the annual membership charge plus the security deposit payable.
- If the renewal option is chosen,
  - LMS asks the member's name and his membership number
    - Checks whether he is a valid member.
  - If the name represents a valid member,
    - the membership expiry date is updated and the annual membership bill is printed,
    - Otherwise an error message is displayed.
- If the cancel membership option is selected and the name of a valid member is entered,
  - the membership is cancelled,
  - a cheque for the balance amount due to the member is printed
  - The membership record is deleted.
- Decision Tree



- Decision Table
- Decision tables specify:
  - which variables are to be tested
  - what actions are to be taken if the conditions are true,
  - The order in which decision making is performed.
- A decision table shows in a tabular form:

- processing logic and corresponding actions

- Upper rows of the table specify:
  - the variables or conditions to be evaluated
- Lower rows specify:
  - The actions to be taken when the corresponding conditions are satisfied.
- In technical terminology,
  - a column of the table is called a rule:
  - A rule implies:
    - If a condition is true, then execute the corresponding action.
- Example:
- Conditions

Valid selection		NO	YES	YES	YES
New member	--	YES	NO	NO	
Renewal	--	NO	YES	NO	
Cancellation		--	NO	NO	YES

- Actions

Display error message	--	--	--	
Ask member's name etc.				
Build customer record	--	--		--
Generate bill	--		--	
Ask membership details	--			
Update expiry date		--	--	--
Print cheque		--	--	--
Delete record	--	--	--	

  - Comparison
- Both decision tables and decision trees
  - can represent complex program logic.
- Decision trees are easier to read and understand
  - when the number of conditions are small.
- Decision tables help to look at every possible combination of conditions.

11.a) What kind of risk is perceived in the software development? How do you handle it effectively? Explain with an example. (6) (Nov 2017)

b) Explain about empirical estimation techniques. (5) (Nov 2017)

Empirical Size Estimation Techniques

Empirical estimation techniques are based on making an educated guess of the project parameters. There are two basic empirical estimation techniques:

- Expert Judgement Technique
- Delphi Cost Estimation

Expert Judgement:

- An euphemism for guess made by an expert.
- Suffers from individual bias.

Delphi Estimation:

- Overcomes some of the problems of expert judgement.

Expert judgement

- Experts divide a software product into component units:
  - e.g. GUI, database module, data communication module, billing module, etc.
- Add up the guesses for each of the components.

Delphi Estimation:

- Team of Experts and a coordinator.
- Experts carry out estimation independently:
  - mention the rationale behind their estimation.
  - coordinator notes down any extraordinary rationale:
    - circulates among experts.
- Experts re-estimate.
- Experts never meet each other to discuss their viewpoints.